```
              context.system.settings
                .config.getInt("example.countwords.workers-per-node")
          for (i <- 0 to numberOfWorkers) {
            context.spawn(Worker(), s"worker-$i")
          }
        }
        Behaviors.receiveMessage {
          case SelfUp(_) =>
            val router = context.spawnAnonymous {                    ❺
              Routers
                .group(Worker.RegistrationKey)
            }
            context.spawn(Master(router), "master")                  ❻
            Behaviors.same
        }
    }
  }
}
```

❶ Only spawns a master with its router if this node has a director role

❷ Subscribes to be notified when Up after the minimum number of members is reached

❸ Only spawns workers if the node is started with an aggregator role

❹ The number of workers to create is taken from the word.conf file.

❺ Spawns a router with a random name

❻ Spawns a router. The master under the notification node director is Up.

Next, let’s examine the master.

8.4.3 Work distribution in the master

This chapter is about communication between different JVMs; therefore, messages sent to another node—here, from the aggregator to the director—must be serialized. For this reason, part of the protocol of the master is defined with `CborSerializable`. You learn more about serialization later, when you have a complete picture of the master and worker implementation. For now, let’s focus on the master’s protocol: